

算法设计与分析 习题课一

任课老师：苏劲松、赖永炫

课程助教：欧阳洋，赵宇豪，龚玉雷

目录

1. 提醒事项 (3min)

2. 第二章部分习题 (45min)

第一章提醒需要掌握的题型

二分查找 + 拓展内容

$O(1)$ 空间子数组换位【旋转数组】

$O(1)$ 空间原地归并

众数问题：哈希表、分治法、排序法

补充题：分治法

1. 作业上交注意事项

作业名称：第3次作业3306_张三.pdf

上传时间：某章讲完的一周内交该章作业

代码要求：最低要求：伪代码；也可以写代码

作业答案：后续会上传到ftp对应目录

2-3 改写二分搜索算法。

请改写二分搜索算法，使得当搜索元素 x 不在数组中时，返回小于 x 的最大元素位置 i 和大于 x 的最小元素位置 j 。设 $a[0:n-1]$ 是已排序的数组。当搜索元素在数组中时， i 和 j 相同，均为 x 在数组中的位置。

```
template<class Type>
```

```
int BinarySearch(Type a[], const Type& x, int n) {
```

// 在 $a[0] \leq a[1] \leq \dots \leq a[n-1]$ 中搜索 x

```
// 找到  $x$  时返回其在数组中的位置，否则返回 -1
```

```
int left = 0;          int right = n-1;
```

```
while (left <= right) {
```

```
    int middle = (left+right)/2;
```

```
    if (x == a[middle])
```

```
        return middle;
```

```
    if (x > a[middle])
```

```
        left = middle+1;
```

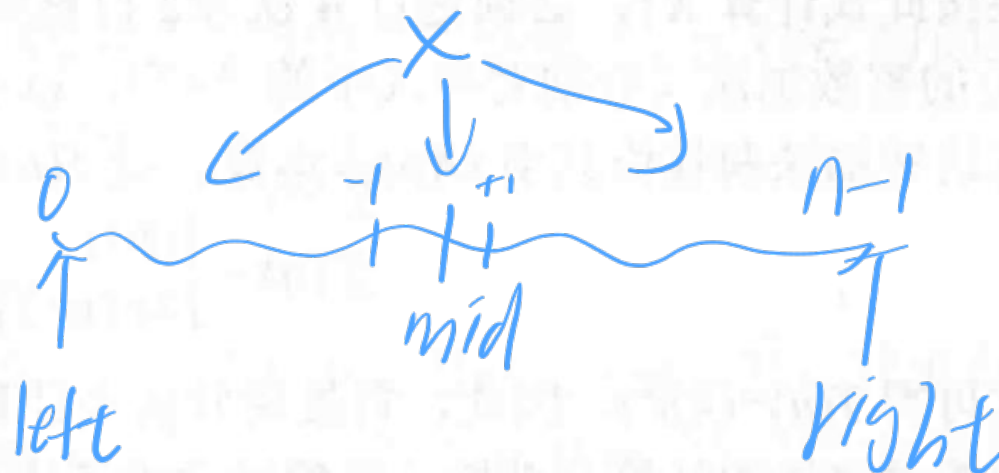
```
    else
```

```
        right = middle-1;
```

```
}
```

```
return -1;
```

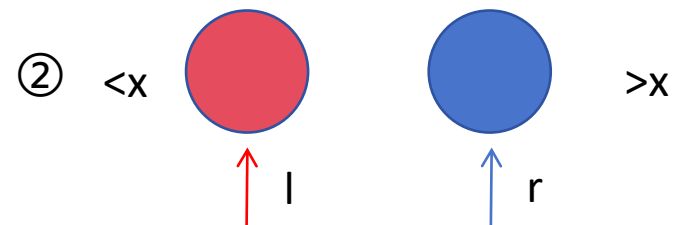
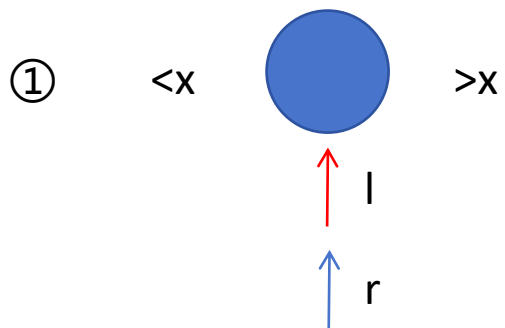
```
}
```

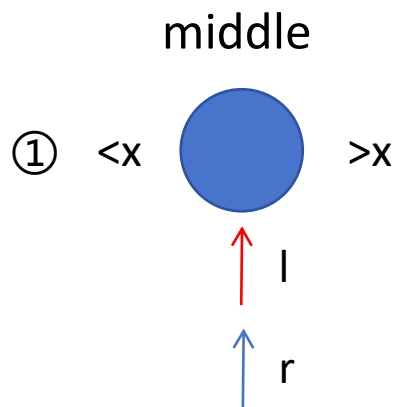


// 未找到 x

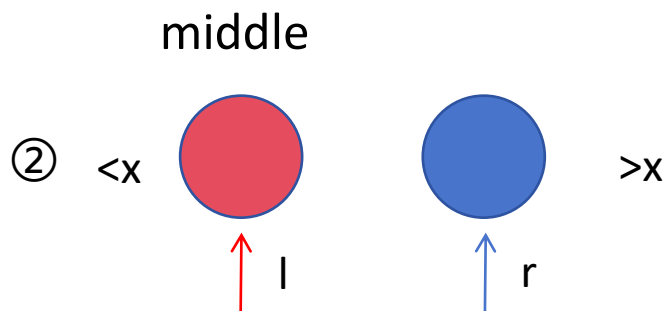
left 和 right 的意义

循环结束前的最后一次迭代，left和right的位置关系只有两种：left=right和left=right-1:





- 在本次迭代中, $middle=l=r$, 若 $a[middle]>x$, $r=middle-1$, 此时 $a[r]<x$, $a[l]=a[middle]>x$ 。l和r就是要找的大于x的最小元素和小于x的最大元素。
- 若 $a[middle]<x$, 则迭代结束后, $l=middle+1$, $a[r]=a[middle]<x$, $a[l]>x$ 。l和r仍是要找的大于x的最小元素和小于x的最大元素



- 在本次迭代中, $middle=l$, 若 $a[middle]>x$, $r=middle-1$, 此时 $a[r]<x$, $a[l]=a[middle]>x$ 。l和r就是要找的大于x的最小元素和小于x的最大元素。
- 若 $a[middle]<x$, 则迭代结束后, $l=middle+1=r$, 回到情况①, 故l和r仍是要找的大于x的最小元素和小于x的最大元素。

结论: 当x不在数组中时left和right的最终值就是大于x的最小元素和小于x的最大元素。

```

template<class T>
int binarySearch(T a[], const T&x, int left, int right, int &i, int &j) {
    int middle;

    while (left <= right) {
        middle = (left + right) / 2;
        if (x == a[middle]) {
            i = j = middle;
            return 1;
        }
        if (x > a[middle])
            left = middle + 1;
        else
            right = middle - 1;
    }
    i = right;
    j = left;
    return 0;
}

```

while ($l < r$)

$m = \frac{l+r}{2}$

if ($x = a[m]$)

$i = j = m$

return 1

if ($x > a[m]$)

$l = m + 1$

else $r = m - 1$

$l = r$
 $j = l$

更简单&更重要的问题：找到第一个大于等于target的数字的下标

<https://leetcode.cn/problems/binary-search/>

<https://leetcode.cn/problems/find-first-and-last-position-of-element-in-sorted-array/description/>

实现中的问题 [编辑]

尽管二分查找的基本思想相对简单，但细节可以令人难以招架 ... — 高德纳^[2]

当乔恩·本特利将二分搜索问题布置给专业编程课的学生时，百分之90的学生在花费数小时后还是无法给出正确的解答，主要因为这些错误程序在面对边界值的时候无法运行，或返回错误结果。^[16]1988年开展的一项研究显示，20本教科书里只有5本正确实现了二分搜索。^[17]不仅如此，本特利自己1986年出版的《编程珠玑》一书中的二分搜索算法存在整数溢出的问题，二十多年来无人发现。Java语言的库所实现的二分搜索算法中同样的溢出问题存在了九年多才被修复。^[18]

参考标库写法：

c++ 的 lower_bound

python的二分查找标准库 bisect_left

```
3
4  a = [2, 3, 4, 7, 7, 7, 8, 10]
5  ans = bisect_left(a, 7) # 第一个大于等于7的元素
6
7  print(ans) # 3
8
9  ans = bisect_right(a, 7) # 第一个大于7的元素
10 print(ans) # 6
11
12 def bisect_left_impl(a, target):
13     l, r = 0, len(a)-1
14     while l <= r:
15         m = (l + r) // 2
16         if a[m] < target:
17             l = m + 1 # 上课的时候填空
18         else: # >=
19             r = m - 1
20     return l
21
22 ans = bisect_left_impl(a, 7) # 3
23 print(ans)
24
```

2-4 大整数乘法的 $O(nm^{\log(3/2)})$ 算法。

给定两个大整数 u 和 v ，它们分别有 m 和 n 位数字，且 $m \leq n$ 。用通常的乘法求 uv 的值需要 $O(mn)$ 时间。可以将 u 和 v 均看作有 n 位数字的大整数，用本章介绍的分治法，在 $O(n^{\log 3})$ 时间内计算 uv 的值。当 m 比 n 小得多时，用这种方法就显得效率不够高。试设计一个算法，在上述情况下用 $O(nm^{\log(3/2)})$ 时间求出 uv 的值。

$n^{\log 3}$

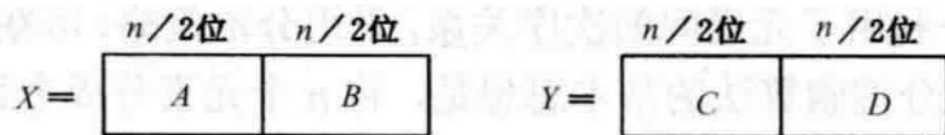


图 2-3 大整数 X 和 Y 的分段

$$XY = AC \times 2^n + ((A-B)(D-C) + AC + BD) \times 2^{n/2} + BD$$

$$XY = AC \times 2^n + (A-B)(D-C) + AC + BD \times 2^{\frac{n}{2}} + BD$$

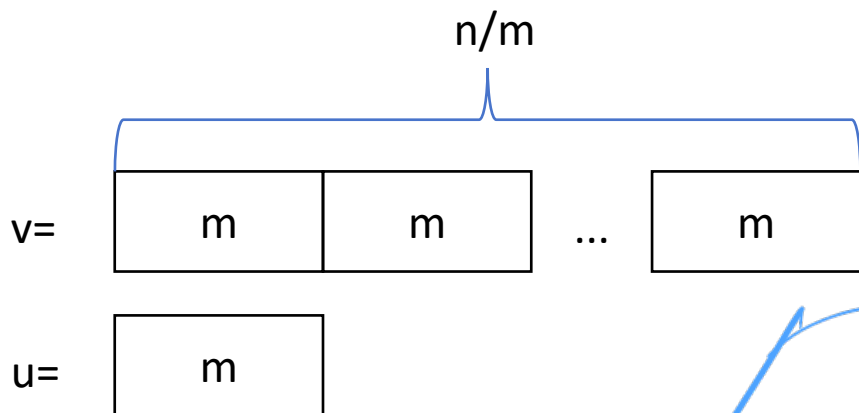
$$T(n) = \begin{cases} O(1) & n=1 \\ 3T(n/2) + O(n) & n>1 \end{cases}$$

$$\longrightarrow T(n) = O(n^{\log 3})$$

$n=1$

$$T(n) = \begin{cases} O(1) & n=1 \\ 3T(n/2) + O(n) & n>1 \end{cases}$$

$u \rightarrow m$
 $v \rightarrow \hat{n}$
 分段



$$T(n) = O(n^{\log 3})$$

一共需要进行 n/m 次 m 位大整数乘法，每次耗时 $O(m^{\log 3})$ ，一共耗时 $O((n/m) m^{\log 3}) = O(nm^{\log(3/2)})$

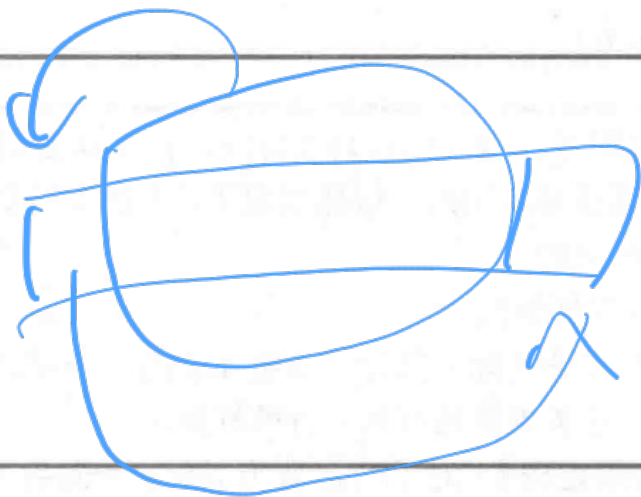
2-8 $O(1)$ 空间子数组换位算法。

设 $a[0:n-1]$ 是有 n 个元素的数组, k ($0 \leq k \leq n-1$) 是一个非负整数。试设计一个算法将子数组 $a[0:k-1]$ 与 $a[k:n-1]$ 换位。要求: 算法在最坏情况下耗时 $O(n)$, 且只用到 $O(1)$ 的辅助空间。

1. 循环换位法: 每次整体移动1位, 移动 $\min\{k, n-k\}$ 次

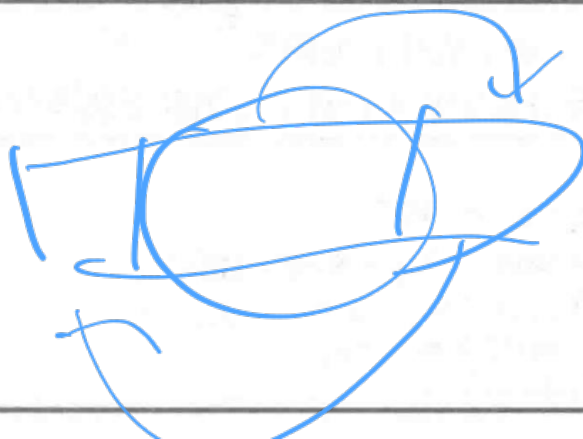
(1) 向前循环换位

```
template<class T>
void forward(T a[], int n, int k) {
    for(int i=0; i < k; i++) {
        T tmp = a[0];
        for(int j=1; j < n; j++)
            a[j-1] = a[j];
        a[n-1] = tmp;
    }
}
```



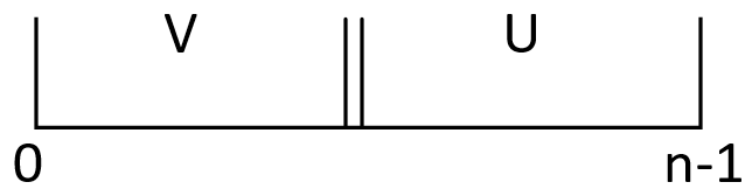
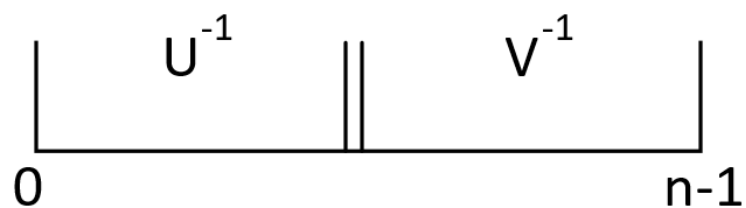
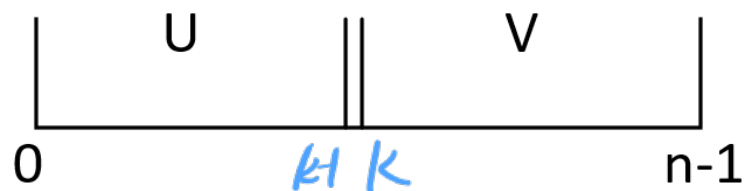
(2) 向后循环换位

```
template<class T>
void backward(T a[], int n, int k) {
    for(int i=k; i < n; i++) {
        T tmp = a[n-1];
        for(int j=n-1; j > 0; j--)
            a[j] = a[j-1];
        a[0] = tmp;
    }
}
```



移动次数为 $\min\{k, n-k\} \times (n+1)$, 最坏情况下时间复杂度为 $O(n^2)$, 空间复杂度为 $O(1)$ 。

2. 三次反转法



将数组划分为 $U=a[0:k-1]$, $V=a[k:n-1]$ 两部分

① 翻转U、V

② 整体翻转

1 2 3 4 5 6 7 8
4 3 2 1 8 7 6 5



时间复杂度: $O(k) + O(n-k) + O(n) = O(n)$,

空间复杂度为 $O(1)$ 。

2-9 $O(1)$ 空间合并算法。

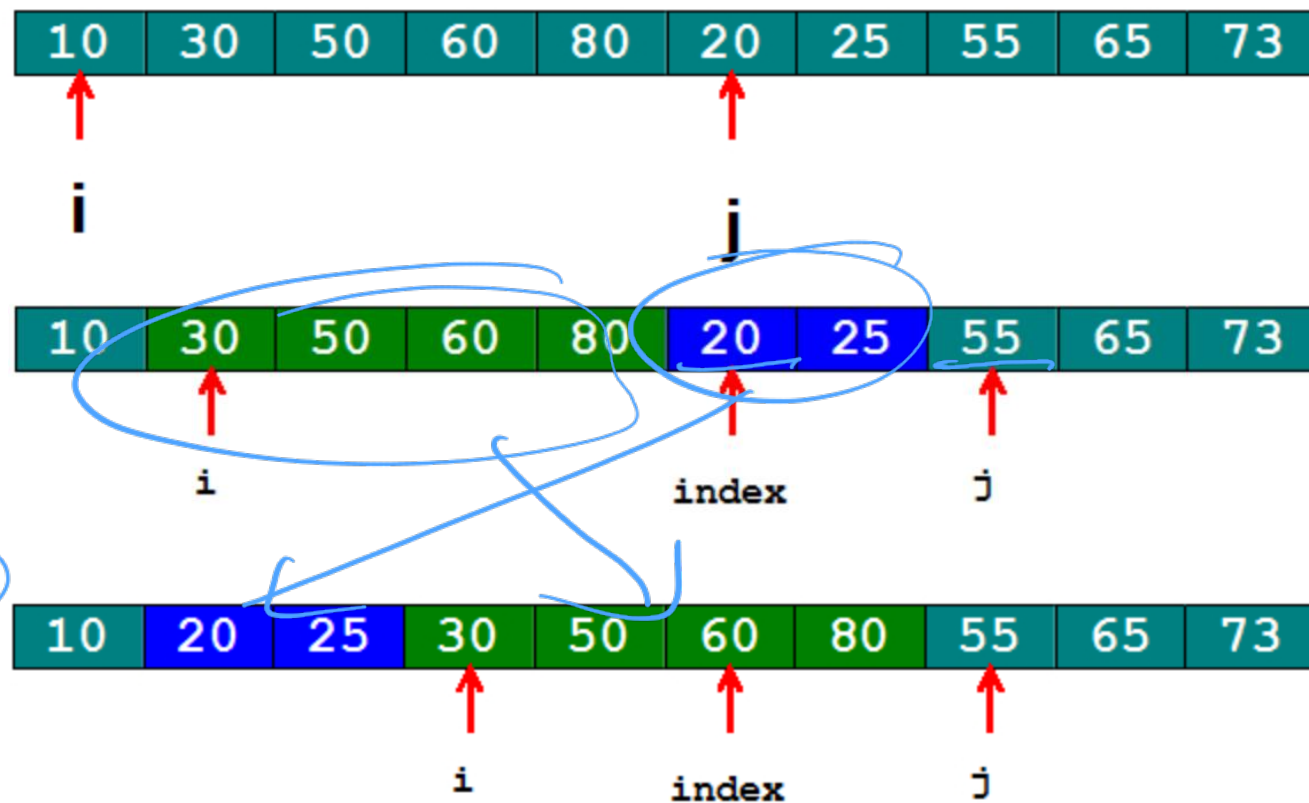
青夏

设子数组 $a[0:k-1]$ 和 $a[k:n-1]$ 已排好序 ($0 \leq k \leq n-1$)。试设计一个合并这两个子数组为排好序的数组 $a[0:n-1]$ 的算法。要求算法在最坏情况下所用的计算时间为 $O(n)$ ，且只用到 $O(1)$ 的辅助空间。

1. 开始时 i, j 指向子数组的第一个元素。

2. i 向后移动，在子数组 $a[0:k-1]$ 中找到第一个比 $a[j]$ 大的数；用临时变量 $index$ 记录 j 的位置，然后 j 向后移动，找到 $a[k:n-1]$ 中第一个大于 $a[i]$ 的数。

3. 用 $O(1)$ 空间子数组换位算法交换 $a[i:k-1]$ 和 $a[index:j-1]$ 。重复操作直到数组有序。



2-1 众数问题。

问题描述：给定含有 n 个元素的多重集合 S ，每个元素在 S 中出现的次数称为该元素的重数。多重集 S 中重数最大的元素称为众数。例如， $S=\{1, 2, 2, 2, 3, 5\}$ 。多重集 S 的众数是 2，其重数为 3。

算法设计：对于给定的由 n 个自然数组成的多重集 S ，计算 S 的众数及其重数。

数据输入：输入数据由文件名为 input.txt 的文本文件提供。文件的第 1 行为多重集 S 中元素个数 n ；在接下来的 n 行中，每行有一个自然数。

结果输出：将计算结果输出到文件 output.txt。输出文件有 2 行，第 1 行是众数，第 2 行是重数。

法1: 哈希表

1.统计元素出现次数：使用哈希表（字典）记录每个元素及其出现的次数。

2.在遍历集合的过程中，跟踪当前的最大重数及其对应的元素。每次更新哈希表后，检查当前元素的重数是否超过已记录的最大值。如果是，则更新最大值和众数。

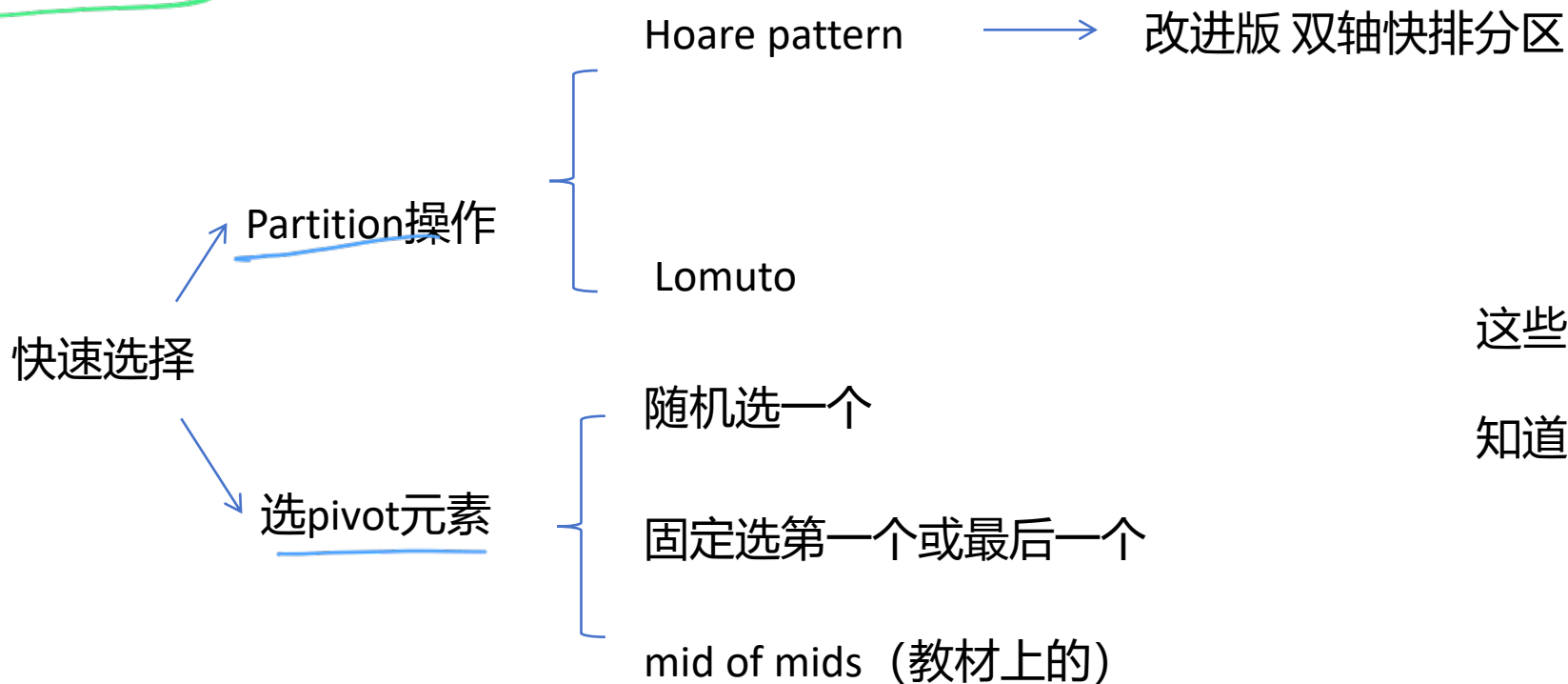
时间复杂度： $O(n)$

空间复杂度： $O(m)$ ， m 为集合中不同元素的数量。

法2: 分治

补充预备知识: 快速选择算法、荷兰国旗 (变种快排partition)

快速选择算法: 在 $O(n)$ 时间找到无序数组的第k小元素, 递归分治算法



这些概念作为了解

知道这个算法的输入输出即可

法2：分治

<https://www.bilibili.com/video/BV1hr4y1y7w2/>

补充预备知识：快速选择算法、**荷兰国旗**（变种快排partition）

荷兰国旗算法：给一个数字 x ，一次遍历，将数组划分为三个区域，小于 x 的，等于 x 的，大于 x 的，返回两个边界

$x = 5$

[| 2 3 5 7 0 1 4 5 6 |]

[2 | 3 5 7 0 1 4 5 6 |]

[2 3 | 5 7 0 1 4 5 6 |]

[2 3 | 5 7 0 1 4 5 6 |]

[2 3 | 5 6 0 1 4 5 | 7]

[2 3 | 5 5 0 1 4 | 6 7]

[2 3 | 5 5 0 1 4 | 6 7]

[2 3 0 | 5 5 1 4 | 6 7]



绿色维护小于 x 区域的右边界，红色维护大于 x 区域的右边界，蓝色是探测指针，当前数字

1. cur数字 $< x$: 当前数字和小区下一个交换，小区右移，当前下一个 $cur++$

2. cur数字等于 x ，cur直接+

3. cur数字大于 x ，cur和大区前一个交换，大区域左移，cur不动

[2 3 0 1 | 5 5 4 | 6 7]

[2 3 0 1 4 | 5 5 | 6 7]

法2: 分治

1. 找出数组的中位数，利用Partition函数将数组划分为小于中位数和大于中位数的两部分。
2. 计算中位数的重数，更新众数。
3. 递归计算左右两部分中位数的重数及更新众数。

```
void mode(int l, int r) {  
    int l1, r1;  
    int med = median(a, l, r);  
    split(a, med, l, r, l1, r1);  
    if (largest < r1 - l1 + 1)  
        largest = r1 - l1 + 1, element = med;  
    if (l1 - l > largest)  
        mode(l, l1 - 1);  
    if (r - r1 > largest)  
        mode(r1 + 1, r);  
}
```

*void mode(int l, int r) {
 int l1, r1;
 int med = median(a, l, r);*

时间复杂度: $O(n \log n)$

空间复杂度: $O(\log n)$

$O(n \log n)$
 $O(\log n)$

2-7 集合划分问题。

问题描述: n 个元素的集合 $\{1, 2, \dots, n\}$ 可以划分为若干非空子集。例如, 当 $n=4$ 时, 集合 $\{1, 2, 3, 4\}$ 可以划分为 15 个不同的非空子集如下:

$\{\{1\}, \{2\}, \{3\}, \{4\}\}$	$\{\{1, 3\}, \{2, 4\}\}$
$\{\{1, 2\}, \{3\}, \{4\}\}$	$\{\{1, 4\}, \{2, 3\}\}$
$\{\{1, 3\}, \{2\}, \{4\}\}$	$\{\{1, 2, 3\}, \{4\}\}$
$\{\{1, 4\}, \{2\}, \{3\}\}$	$\{\{1, 2, 4\}, \{3\}\}$
$\{\{2, 3\}, \{1\}, \{4\}\}$	$\{\{1, 3, 4\}, \{2\}\}$
$\{\{2, 4\}, \{1\}, \{3\}\}$	$\{\{2, 3, 4\}, \{1\}\}$
$\{\{3, 4\}, \{1\}, \{2\}\}$	$\{\{1, 2, 3, 4\}\}$
$\{\{1, 2\}, \{3, 4\}\}$	

算法设计: 给定正整数 n , 计算出 n 个元素的集合 $\{1, 2, \dots, n\}$ 可以划分为多少个不同的非空子集。

stirling(n, k) 函数：计算n个数划分为k个集合的分法数量。

k=1或者k=n时，都只有一种划分。

将第n个数作为一个子集，则剩下的n-1个数要划分k-1个集合，即stirling(n-1, k-1)；将第n个数加入到其他集合，则剩下的n-1个数要划分k个集合，且第n个数可以在这k个集合中随便选择，即 $k * \text{stirling}(n-1, k)$ 。

```
function stirling(n, k):  
    if k == 1 or k == n:  
        return 1  
    else:  
        return stirling(n-1, k-1) + k * stirling(n-1, k)
```

// 计算贝尔数 B(n)

```
function bell(n):  
    sum = 0  
    for k = 1 to n:  
        sum += stirling(n, k)  
    return sum
```

$(1 \ 2 \ 3 \ 4 \ \dots \ n-1) | n)$

补充题:

设 $T[1:n]$ 是一个含有 n 个元素的数组。如果元素 x 的出现次数超过 $n/2$, 称元素 x 为数组 T 的主元素。

(1) 如果这 n 个元素存在序关系, 比如 n 个整数

(2) 如果这 n 个元素不存在序关系, 比如 n 个坐标

请分别针对上述两种情况, 分别设计时间复杂度为 $O(n)$ 的分治算法, 判断该数组里是否有主元素。

(1) 若x是T的主元素，则x一定是T的中位数。

1. 利用线性时间选择算法找到T的中位数median
2. 遍历数组，计算median的重数，通过重数判断median是否是主元素

```
int master(T)
{
    int median = Select(T, 1, n, (n+1)/2); //求中位数
    int cnt = 0
    //计算中位数出现次数
    for (int i = 1; i <= n; i++)
        if (T[i] == median) cnt++;
    if (cnt > n/2)
        return median
    else return -1;
}
```

$median = select(T, 1, n, \frac{n+1}{2})$

时间复杂度：线性时间选择 $O(n)$ + 计算重数 $O(n) = O(n)$

(2) 摩尔投票算法：将不同的元素两两抵消，最终剩下的元素就有可能是主元素。

1. 如果票数为0，将当前元素设为候选元素，并将票数设置为1。
2. 如果当前元素等于候选元素，则票数加1。
3. 如果当前元素不等于候选元素，则票数减1。
4. 遍历数组，判断候选元素是否为主元素。

时间复杂度为 $O(n)$ ，空间复杂度为 $O(1)$

```
function findMajorityElement(nums):  
    candidate = None  
    count = 0  
  
    for num in nums:  
        if count == 0:  
            candidate = num  
        if candidate == num:  
            count += 1  
        else:  
            count -= 1  
  
    // 进行第二次遍历，验证 candidate 是否为主元素  
    count = 0  
    for num in nums:  
        if num == candidate:  
            count += 1  
  
    if count > len(nums) / 2:  
        return candidate  
    else:  
        return None
```

分治版本：函数求数组最终候选元素和票数。

分：将数组从中间分为左右两部分，递归地在左右两部分分别找出各自的候选元素和票数。当数组只有一个元素时，候选元素就是该元素，票数为1。

治：若左右子数组的候选元素相同，则该元素也是数组的候选元素，票数为左右票数之和。若不相同，则候选元素取票数较多的，票数为左右票数之差。


```

function findMajorityUnordered(nums):
    if nums is empty:
        return False
    candidate, _ = divideAndConquerVote(nums, 0, length(nums) - 1)
    count = 0
    for num in nums:
        if num == candidate:
            count += 1
    return count > length(nums) / 2

function divideAndConquerVote(nums, left, right):
    if left == right:
        return (nums[left], 1) // 只有一个元素，候选元素是自己，票数为1

    mid = (left + right) / 2
    (left_cand, left_count) = divideAndConquerVote(nums, left, mid)
    (right_cand, right_count) = divideAndConquerVote(nums, mid + 1, right)

    if left_cand == right_cand:
        return (left_cand, left_count + right_count) // 相同元素，票数相加

    return (left_cand, left_count - right_count) if left_count > right_count
        else (right_cand, right_count - left_count)

```

$$T(n) = 2T(n/2) + O(1)$$

时间复杂度： $O(\log n) + O(n) = O(n)$ ，空间复杂度 $O(\log n)$

2-5 5次 $n/3$ 位整数的乘法。

在用分治法求两个 n 位大整数 u 和 v 的乘积时, 将 u 和 v 都分割为长度为 $n/3$ 位的 3 段。证明可以用 5 次 $n/3$ 位整数的乘法求得 uv 的值。按此思想设计一个求两个大整数乘积的分治算法, 并分析算法的计算复杂性 (提示: n 位的大整数除以一个常数 k 可以在 $\theta(n)$ 时间内完成。符号 θ 所隐含的常数可能依赖于 k)。

Toom-Cook 算法: 将两个 n 位大整数 u 、 v 切割成长度为 n/m 位的 m 段, 可以用 $2m-1$ 次 n/m 位整数乘法计算 uv 。

事实上, 设 $x=2^{n/m}$, 可以将 u 和 v 及其乘积 $w=uv$ 表示为

$$u=u_0+u_1x+\cdots+u_{m-1}x^{m-1}$$

$$v=v_0+v_1x+\cdots+v_{m-1}x^{m-1}$$

$$w=uv=w_0+w_1x+w_2x^2+\cdots+w_{2m-2}x^{2m-2}$$

将 u 、 v 和 w 都看作关于变量 x 的多项式, 并取 $2m-1$ 个不同的数 $x_1, x_2, \cdots, x_{2m-1}$ 代入多项式, 可得

$$u(x_i)=u_0+u_1x_i+\cdots+u_{m-1}x_i^{m-1}$$

$$v(x_i)=v_0+v_1x_i+\cdots+v_{m-1}x_i^{m-1}$$

$$w(x_i)=u(x_i)v(x_i)=w_0+w_1x_i+w_2x_i^2+\cdots+w_{2m-2}x_i^{2m-2}$$

$$\begin{bmatrix} w(x_1) \\ w(x_2) \\ \vdots \\ w(x_{2m-1}) \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{2m-2} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{2m-2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{2m-1} & x_{2m-1}^2 & \cdots & x_{2m-1}^{2m-2} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{2m-2} \end{bmatrix}$$

设

$$B = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{2m-2} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{2m-2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{2m-1} & x_{2m-1}^2 & \cdots & x_{2m-1}^{2m-2} \end{bmatrix} \quad (|B| \text{ 为范德蒙德行列式, } B \text{ 必可逆})$$

则

$$\begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{2m-2} \end{bmatrix} = B^{-1} \begin{bmatrix} w(x_1) \\ w(x_2) \\ \vdots \\ w(x_{2m-1}) \end{bmatrix}$$

$$w(x_i) = u(x_i)v(x_i) = (u_0 + u_1x_i + \cdots + u_{m-1}x_i^{m-1}) * (v_0 + v_1x_i + \cdots + v_{m-1}x_i^{m-1})$$

n/m位整数

n/m位整数

计算 w_i 只需要一次2个n/m位整数乘法, 所以一共需要 $2m-1$ 次n/m位整数乘法。

下面用 $m=3$ 的具体例子来说明。设 $x=2^{n/3}$ ，可以将 u 和 v 及其乘积 $w=uv$ 表示为

$$u=u_0+u_1x+u_2x^2$$

$$v=v_0+v_1x+v_2x^2$$

$$w=uv=w_0+w_1x+w_2x^2+w_3x^3+w_4x^4$$

取 5 个数 $x_1, x_2, \dots, x_5$ 如下: $x_1=0, x_2=-2, x_3=2, x_4=-1, x_5=1$ ，代入多项式得

$$a=w(x_1)=u_0v_0$$

$$b=w(x_2)=(u_0-2u_1+4u_2)(v_0-2v_1+4v_2)$$

$$c=w(x_3)=(u_0+2u_1+4u_2)(v_0+2v_1+4v_2)$$

$$d=w(x_4)=(u_0-u_1+u_2)(v_0-v_1+v_2)$$

$$e=w(x_5)=(u_0+u_1+u_2)(v_0+v_1+v_2)$$

$$\begin{bmatrix} a \\ b \\ c \\ d \\ e \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & -2 & 4 & -8 & 16 \\ 1 & 2 & 4 & 8 & 16 \\ 1 & -1 & 1 & -1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix}$$

$$\begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & -2 & 4 & -8 & 16 \\ 1 & 2 & 4 & 8 & 16 \\ 1 & -1 & 1 & -1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} a \\ b \\ c \\ d \\ e \end{bmatrix}$$

解得

$$w_0 = a$$

$$w_1 = \frac{b-c-8(d-e)}{12}$$

$$w_2 = \frac{-b-c+16(d+e)-30a}{24}$$

$$w_3 = \frac{-b+c+2(d-e)}{12}$$

$$w_4 = \frac{b+c-4(d+e)+6a}{24}$$

按此分解设计的求两个 n 位大整数乘积的分治算法需要 5 次 $n/3$ 位整数乘法。分割及合并步所需的加减法和数乘运算时间为 $O(n)$ 。设 $T(n)$ 是算法所需的计算时间，则

$$T(n) = \begin{cases} O(1) & n=1 \\ 5T(n/3) + O(n) & n>1 \end{cases}$$

$$T(n) = O(n^{\log_3 5})$$